A REPORT

ON

DOCUMENT TRACKING SYSTEM

By

Alok Raj	AP24110010335
Jayesh Krishna	AP24110010415
Raushan Kumar	AP24110010465
Satwik Gupta	AP24110010470
Purva Patel	AP24110010486

Prepared in the partial fulfillment of the
Project Based Learning of Course CSE 211 – Full Stack Development

SRM UNIVERSITY, AP

APRIL 2026

Contents

Abstract **1**

1 Introduction **1**

2 Problem Statement **1**

3 Scenario Based Case Study **2**

4 System Requirements **2**

4.1 Software Requirements 2

4.2 Hardware Requirements 3

5 Project Architecture **3**

5.1 Technical Architecture 3

5.2 Workflow Architecture 4

5.2.1 Overview 4

5.2.2 Upload 4

5.2.3 Visibility Rules 4

5.2.4 Update Document Details 5

5.2.5 Workflow Actions 5

5.2.6 Remarks 6

5.2.7 Data Refresh 6

5.2.8 History and Audit Trail 6

5.3 ER Diagram 7

6 Key Features **7**

7 Roles and Responsibilities **8**

7.1 Employee 8

7.2 Manager 8

7.3 Admin 8

8 User Flow **9**

8.1 Employee Flow 9

8.2 Manager Flow 9

8.3 Admin Flow 9

9 MVC Pattern **9**

9.1 Model Layer 9

9.2 Controller Layer 9

9.3 View Layer (Routing Layer) 10

9.4 Advantages of Using MVC in This Project 10

10 Project Setup and Configuration **10**

10.1 Creating the Project Structure 10

10.2 Frontend Setup 12

10.3 Backend Setup 13

10.4 Environment Variables 14

11 Backend Development **14**

11.1 Backend Structure 14

11.2 Backend Modules 15

12 Database Development **15**

12.1 MongoDB Configuration 15

12.2 Schemas and Models 15

12.2.1 `User.js` 16

12.2.2 `Document.js` 16

12.2.3 `Workflow.js` 16

12.2.4 `DocumentLog.js` 17

12.3 Relationships and Database Integrity 17

13 Front-End Development **17**

13.1 Frontend Structure 17

13.2 Frontend Pages and Components 18

14 API Overview **19**

14.1 Authentication Endpoints 19

14.2 Document Endpoints 19

14.3 Workflow and User Endpoints 19

15 Advantages of the Implemented System **19**

16 Limitations and Future Scope **20**

17 Output Screens **20**

17.1 Landing Page 21

17.2 Login 21

17.3 Signup 22

17.4 Dashboard 22

17.5 Upload Document 23

17.6 Chatbot 23

17.7 Admin Panel 24

17.8 Document Detail 24

18 Conclusion **24**

Abstract

The **Document Tracking System** is a full-stack web application developed to manage documents from submission to final decision in a transparent, secure, and auditable way. The platform supports document upload, department-based routing, manager review, forwarding, approval, rejection, audit logging, OTP-based login, Cloudinary file storage, and email notifications. The project addresses common organizational problems such as lack of traceability, delayed approvals, unclear responsibility, and fragmented document handling across departments.

The system is implemented using React, Vite, Node.js, Express.js, MongoDB, Mongoose, JWT, Cloudinary, and Nodemailer. It follows an MVC-style backend design and supports role-based access for employees, managers, and administrators. This report presents the project in a structured format based on the requested template, while keeping the content specific to the Document Tracking System.

1 Introduction

The Document Tracking System is a centralized web platform that helps organizations manage the movement of official documents across departments. In many offices, documents are still handled through email chains, messaging applications, printed files, or spreadsheets. These methods often create confusion about the current status of a document, the person responsible for the next action, and whether the document has been approved, rejected, or forwarded.

This project solves that problem by introducing a digital workflow where every document is uploaded with metadata, assigned to a department or reviewer, and tracked throughout its lifecycle. The system provides controlled access based on user role, maintains a log of workflow actions, and updates users through notifications. As a result, the platform improves accountability, reduces delays, and makes document handling easier for both staff and administrators.

2 Problem Statement

Organizations need a reliable way to submit, review, forward, approve, and reject documents without losing visibility or control. In the absence of a dedicated workflow platform, documents often suffer from the following issues:

- the current review stage is difficult to identify;
- ownership and responsibility are unclear when a file moves between departments;
- approval and rejection decisions are not consistently recorded with remarks;
- audit history is either incomplete or completely missing;
- manual follow-up delays decision-making; and
- sensitive files may be exposed without proper access control.

The problem addressed by this project is therefore the lack of a secure, role-based, workflow-driven, and auditable system for organizational document management. The implemented solution directly addresses this need by digitizing the complete document lifecycle.

3 Scenario Based Case Study

Consider an employee named **Ananya**, who works in the Finance department and needs to submit a reimbursement approval document. In a traditional environment, Ananya may send the file through email and then wait for repeated follow-ups to know whether the manager has seen it, forwarded it, or approved it.

With the Document Tracking System, Ananya logs in securely using OTP verification and uploads the document with a title, description, department, and file attachment. The backend stores the file in Cloudinary, creates a document record, initializes a workflow entry, and records the upload action in the audit log. If a manager exists for the selected department, the document is assigned automatically.

The department manager reviews the document on the dashboard. If additional review is required, the manager forwards it to another user or department with remarks. If the document is valid, the manager approves it; otherwise, it can be rejected with a reason. Each action updates the current status, refreshes the frontend data, and adds a new entry to the audit history. Ananya can now track the document's progress without confusion, and administrators can monitor the entire process centrally.

This case study shows how the system improves transparency, reduces manual coordination, and ensures that each document has a clear workflow path.

4 System Requirements

To ensure smooth development, testing, and execution of the Document Tracking System, the following software and hardware requirements are recommended.

4.1 Software Requirements

These are the essential tools and platforms required to develop, test, and run the application effectively.

- **Operating System:** Windows 10/11, macOS, or Linux for cross-platform development and deployment.
- **Node.js (v16 or above):** Required for frontend tooling and backend runtime.
- **npm (v8 or above):** Used to install and manage project dependencies.
- **React 18:** Builds the interactive frontend user interface.
- **Vite:** Provides fast frontend development and build tooling.
- **Express.js:** Handles REST API creation and backend routing.
- **MongoDB:** Stores application data such as users, documents, workflows, and logs.
- **Mongoose:** Defines schemas and data access logic for MongoDB.
- **Browser:** Latest version of Google Chrome or Mozilla Firefox for UI testing.
- **Postman:** Useful for API development and endpoint verification.
- **Visual Studio Code:** Recommended editor with integrated terminal and Git support.

- **Cloudinary Account:** Required for storing uploaded document files.
- **SMTP Email Account:** Required for OTP delivery and notification emails.

4.2 Hardware Requirements

The following specifications are suitable for development and normal usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) or AMD Ryzen 5 or better.
- **RAM:** Minimum 8 GB, with 16 GB recommended for smoother multitasking.
- **Storage:** At least 1 GB of free space for dependencies, database files, and project assets.
- **Display:** 1366×768 or higher for comfortable coding and testing.
- **Internet Connection:** Required for package installation, Cloudinary usage, and email configuration.

5 Project Architecture

The Document Tracking System follows a layered full-stack architecture in which the frontend handles user interaction, the backend manages business logic and workflow processing, the database stores structured records, and external services support file storage and email delivery.

5.1 Technical Architecture

The technical architecture is organized into clear layers so that the user interface, backend processing, data storage, and external services remain easy to understand and maintain. A user works through the React frontend, which communicates with the Express backend through REST APIs. The backend handles authentication, validation, workflow actions, and audit logging. MongoDB stores persistent records, Cloudinary stores uploaded files, and the SMTP mail service is used for OTP delivery and workflow notifications.

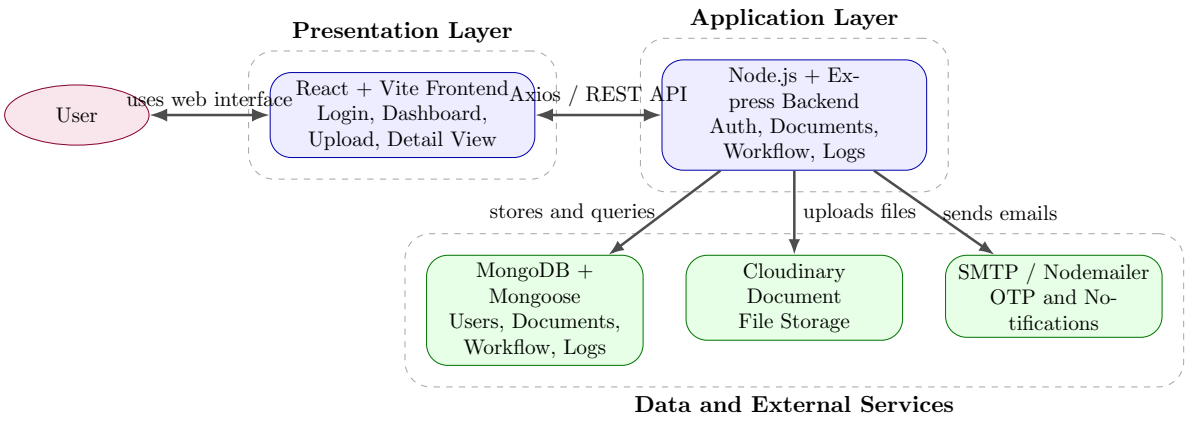

Figure 1: Technical architecture of the Document Tracking System

5.2 Workflow Architecture

Document Workflow. This workflow explains how the application currently manages a document from upload to final decision. The system tracks a document through upload, review, forwarding, approval, and rejection, while keeping the active workflow state and audit history synchronized.

5.2.1 Overview

There are three main roles involved in the workflow.

- `employee`: can upload documents and view documents they uploaded or that are currently assigned to them;
- `manager`: can view documents in their own department, documents assigned to them, and documents they uploaded; and
- `admin`: can access all documents.

Each document is represented by the following records.

- a `Document` record for metadata such as title, description, file, department, status, and remarks;
- a `Workflow` record for the current workflow stage and current assignee; and
- `DocumentLog` entries for the action history.

5.2.2 Upload

When a user uploads a document, the backend performs the following steps.

1. a new `Document` record is created;
2. the document department is resolved from either the submitted department or the uploader's department;
3. a `Workflow` record is created with `currentStage = Submitted`;
4. the workflow is assigned to the default manager of that department when one exists; and
5. a `DocumentLog` entry is created with the action `Upload`.

After upload, the document status is `Submitted`, the workflow stage is `Submitted`, and the document is usually assigned to the manager of the selected department.

5.2.3 Visibility Rules

The system enforces role-based visibility and action control.

- **Who can see a document:**
  - `admin`: can see every document;

  - `employee`: can see documents they uploaded or documents currently assigned to them; and
  - `manager`: can see documents they uploaded, documents currently assigned to them, or documents in their own department.

- **Who can act on workflow:**
  - `admin`;
  - the manager of the document's current department; and
  - the currently assigned user.

- **Uploader UI restriction:** on the document detail page, the uploader should not see `Update Document Details` or `Workflow Action`. This restriction is enforced in the frontend UI for the uploader view.

5.2.4 Update Document Details

Metadata updates are handled separately from workflow actions. Possible updates include title, description, remarks, uploaded file, and department.

- the uploader can still remain the document owner, but the uploader-facing UI hides the update section after upload;
- department changes are restricted to `admin`, the department manager, or the currently assigned user; and
- when the department changes, the workflow assignee is reset to the manager of the new department if one exists.

5.2.5 Workflow Actions

The system supports three workflow actions: `Forward`, `Approve`, and `Reject`.

Forward When a document is forwarded, the document status becomes `Under Review` and the workflow stage becomes `Under Review`. If an assignee is selected, the target department is automatically derived from that assignee's department, users cannot manually change the target department in the UI, and the backend also enforces the same rule. If no assignee is selected, the provided target department is used and the workflow is assigned to that department's manager when one exists. A `DocumentLog` entry is created with the action `Forward`.

Approve When a document is approved, the document status becomes `Approved`, the workflow stage becomes `Approved`, and a `DocumentLog` entry is created with the action `Approve`.

Reject When a document is rejected, the document status becomes `Rejected`, the workflow stage becomes `Rejected`, and a `DocumentLog` entry is created with the action `Reject`.

5.2.6 Remarks

Remarks are stored on the `Document` record. They can be updated during metadata changes and can also be updated while performing a workflow action. This means the latest submitted remark is preserved on the document itself.

5.2.7 Data Refresh

Whenever a document is uploaded, updated, forwarded, approved, rejected, or deleted, the backend updates the document, workflow, and log records through REST endpoints. The document detail page refreshes after user actions and reloads both the document data and the document history.

5.2.8 History and Audit Trail

Every important workflow step creates a `DocumentLog` entry. Common actions include `Upload`, `Forward`, `Approve`, and `Reject`. The document detail page displays this history inside the `Document History` table.

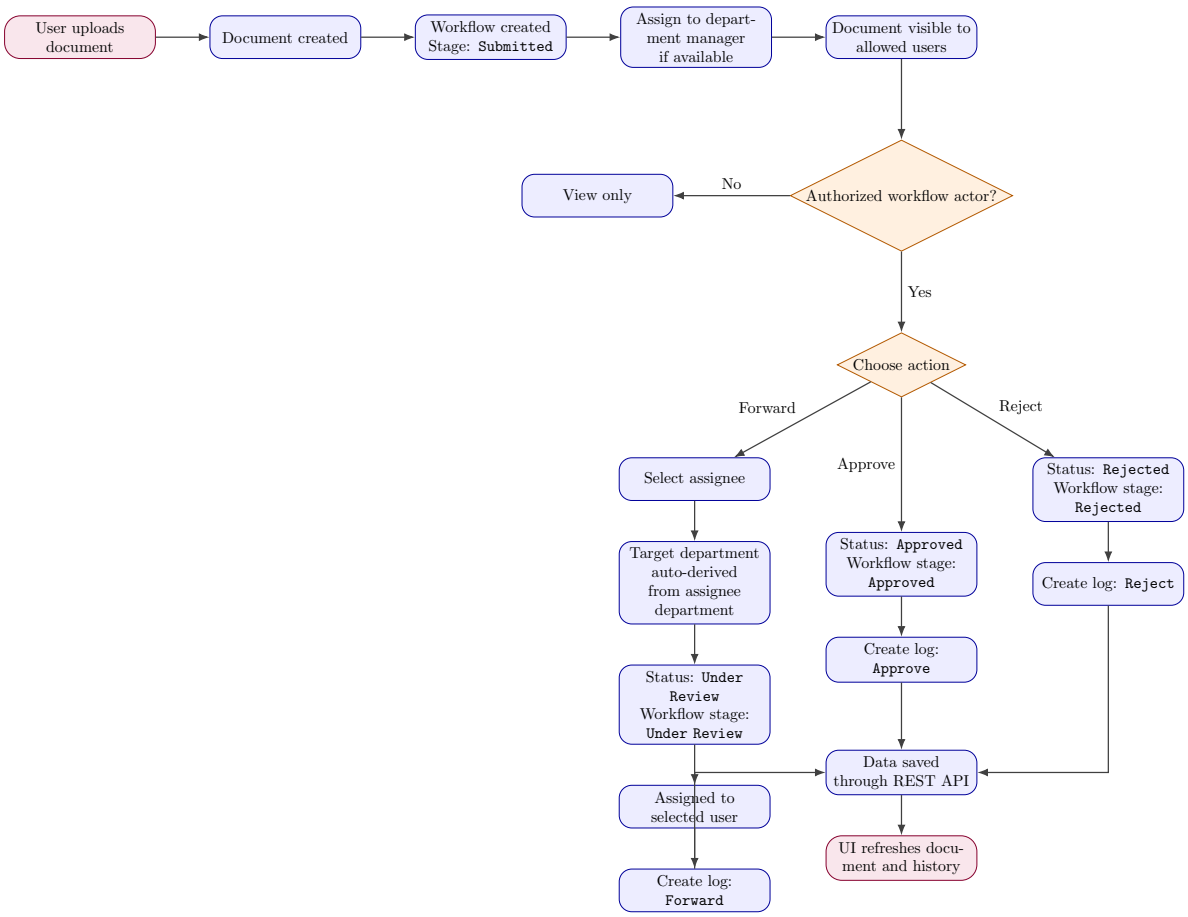

Figure 2: Workflow architecture of the Document Tracking System

5.3 ER Diagram

The implemented database schema uses four Mongoose models: `User`, `Document`, `Workflow`, and `DocumentLog`. These entities capture both the current state of a document and the complete audit trail of actions performed on it. The ER diagram below is based directly on the fields and references present in the actual backend schemas.

- **User** stores account identity, authentication fields, role, department, and OTP data.
- **Document** stores document metadata, file storage information, workflow status, and the uploader reference.
- **Workflow** stores the live routing state of a document, including current stage and assignee.
- **DocumentLog** stores immutable history entries for upload, forward, approve, and reject actions.

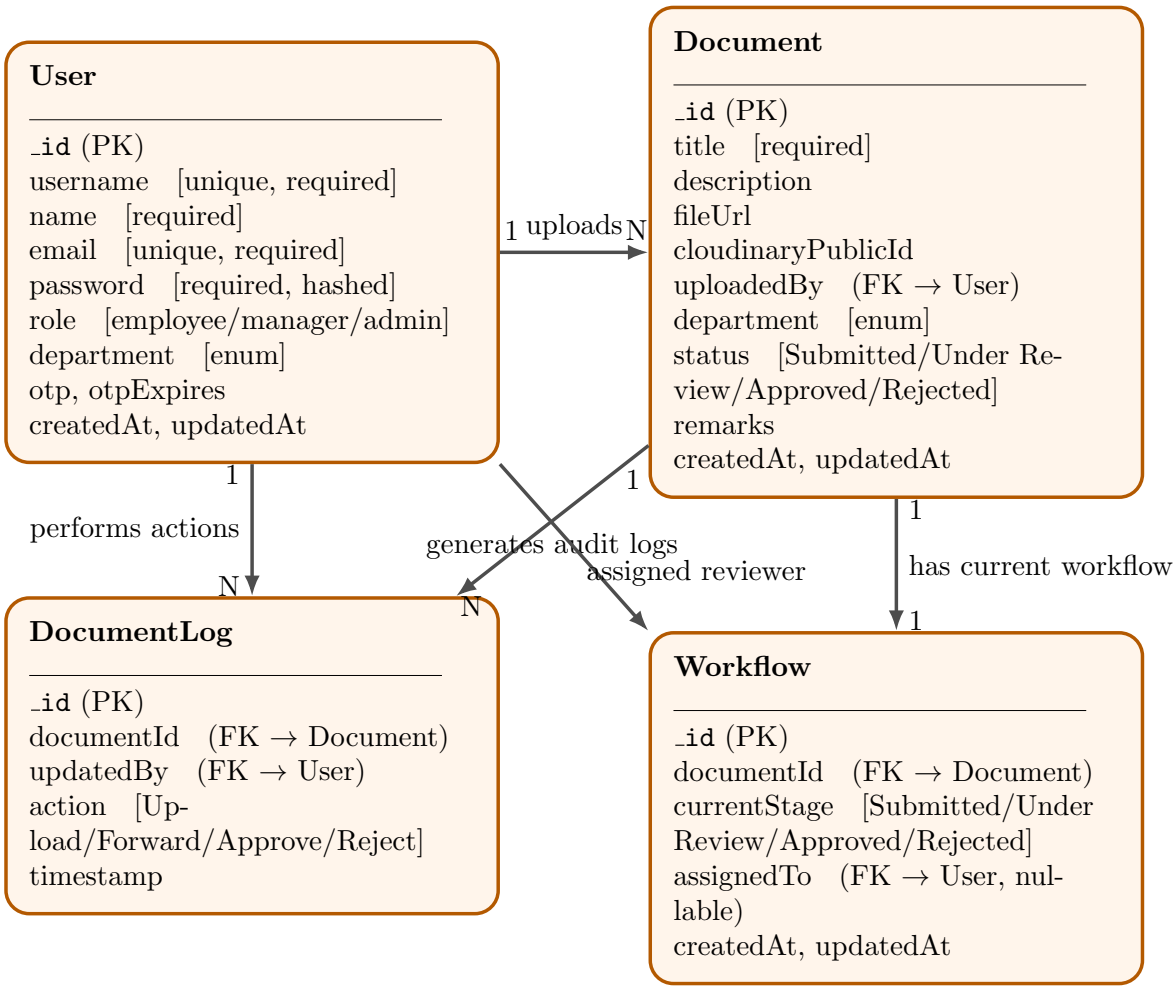

Figure 3: ER diagram derived from the implemented Mongoose schemas

6 Key Features

The major features of the Document Tracking System are listed below.

- user registration and secure login with email OTP verification;
- JWT-based protected authentication for frontend and backend communication;
- role-based access for `employee`, `manager`, and `admin`;
- document upload with title, description, department, and file;
- Cloudinary-based storage for uploaded files;
- status management through `Submitted`, `Under Review`, `Approved`, and `Rejected`;
- workflow forwarding to other users or departments;
- remarks support during update, review, approval, and rejection;
- audit trail for upload and workflow actions;
- search and filter support on the dashboard; and
- admin panel for user management and system-level visibility.

7 Roles and Responsibilities

The platform supports three major roles, each with clearly defined responsibilities.

7.1 Employee

- register and log in using valid credentials and OTP verification;
- upload documents with correct metadata and department selection;
- view documents they uploaded or documents assigned to them;
- track status changes, remarks, and audit history; and
- communicate follow-up context through appropriate document details and remarks.

7.2 Manager

- review documents assigned to them or belonging to their department;
- forward documents to another user or department when further review is needed;
- approve or reject documents with clear remarks;
- monitor department-level workflow progress; and
- ensure that decisions are taken within the proper review path.

7.3 Admin

- manage system-wide access and user records;
- monitor documents, workflows, and logs across all departments;
- support system governance, account management, and issue resolution; and
- maintain the overall integrity, transparency, and security of the application.

8 User Flow

8.1 Employee Flow

- create an account and log in with OTP verification;
- upload a document with metadata and file attachment;
- monitor the document on the dashboard;
- open document details to see workflow stage and history; and
- receive updates as the document moves through the process.

8.2 Manager Flow

- log in and view assigned or department documents;
- inspect document details, metadata, and remarks;
- forward the document for additional review if required;
- approve or reject the document based on authority; and
- trigger status changes that are reflected in logs and notifications.

8.3 Admin Flow

- log in with administrative privileges;
- manage users and access roles;
- monitor documents, workflows, and system activity;
- view audit logs for compliance and troubleshooting; and
- maintain a centralized overview of all departments.

9 MVC Pattern

The backend of the Document Tracking System follows the Model-View-Controller concept, which separates data logic, request handling, and route exposure into clearly defined layers.

9.1 Model Layer

The Model layer handles all data-related logic. Mongoose schemas define the structure of users, documents, workflows, and logs. These models are responsible for storing, retrieving, and updating application data inside MongoDB.

9.2 Controller Layer

The Controller layer contains the core business logic. It processes incoming requests, performs validation and authorization checks, calls model methods, updates workflow state, and sends

responses back to the client. Controllers such as `authController.js`, `documentController.js`, `workflowController.js`, `userController.js`, and `logController.js` belong to this layer.

9.3 View Layer (Routing Layer)

In this REST-based backend, the view layer is represented by the routes. Route files define the available endpoints and connect HTTP methods to the appropriate controller functions. These routes determine how the application responds to actions such as registration, login, upload, workflow update, and log retrieval.

9.4 Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a specific responsibility, which improves readability and maintenance.
- **Scalability:** New modules can be added with minimal impact on the rest of the system.
- **Reusability:** Logic in controllers and models can be reused across different routes.
- **Testing:** Controllers and models can be tested independently.
- **Team Collaboration:** Multiple developers can work on models, controllers, and routes in parallel.

10 Project Setup and Configuration

10.1 Creating the Project Structure

The project is organized into separate frontend and backend folders for clarity and maintainability.

1. create the main project folder for the Document Tracking System;
2. create two main subfolders: `frontend` and `backend`;
3. open the project in Visual Studio Code; and
4. initialize the frontend and backend independently according to their technology stacks.

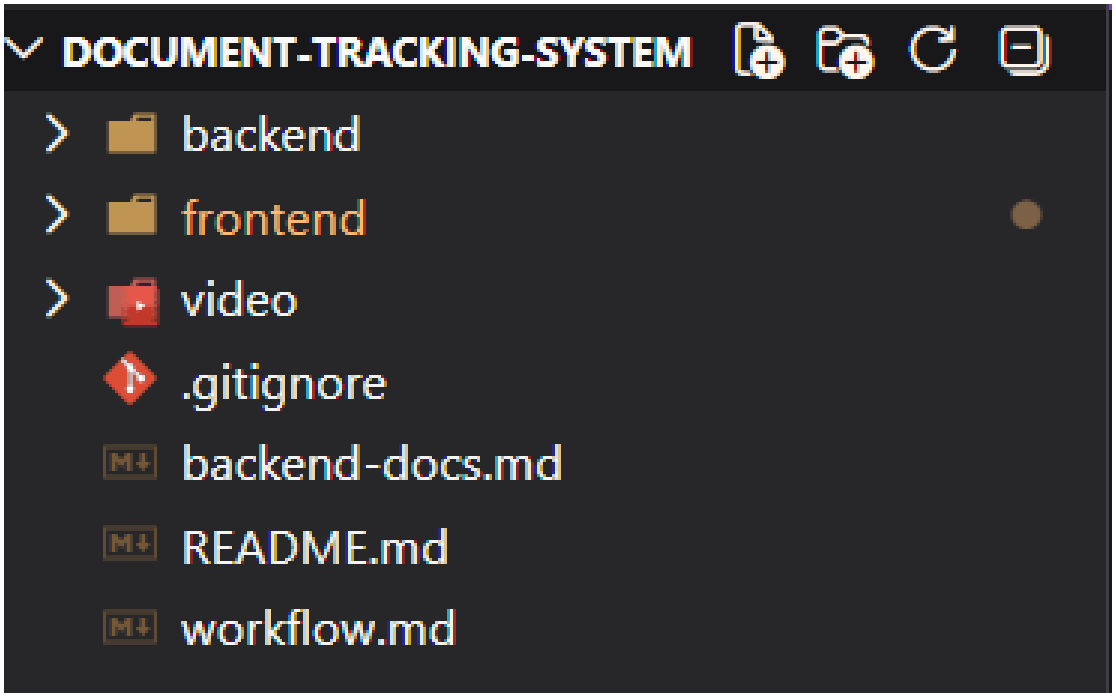

Figure 4: Initial project structure with separate frontend and backend folders

10.2 Frontend Setup

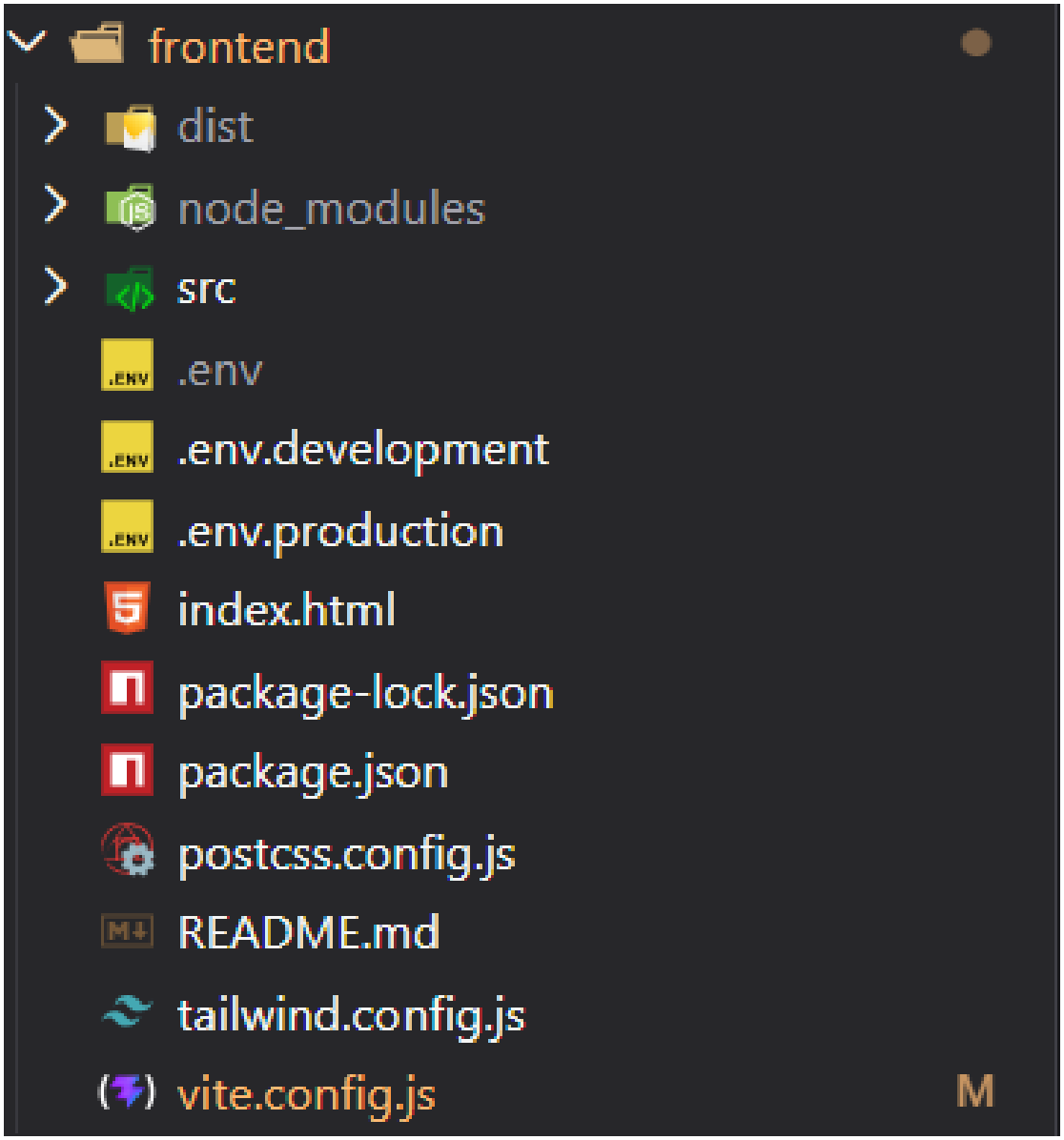

Figure 5: Frontend setup steps and development server configuration

```
cd frontend
npm install
npm run dev
```

The frontend uses Vite with React. During development, it can be configured to use:

```
VITE_API_BASE_URL=/api
```

10.3 Backend Setup

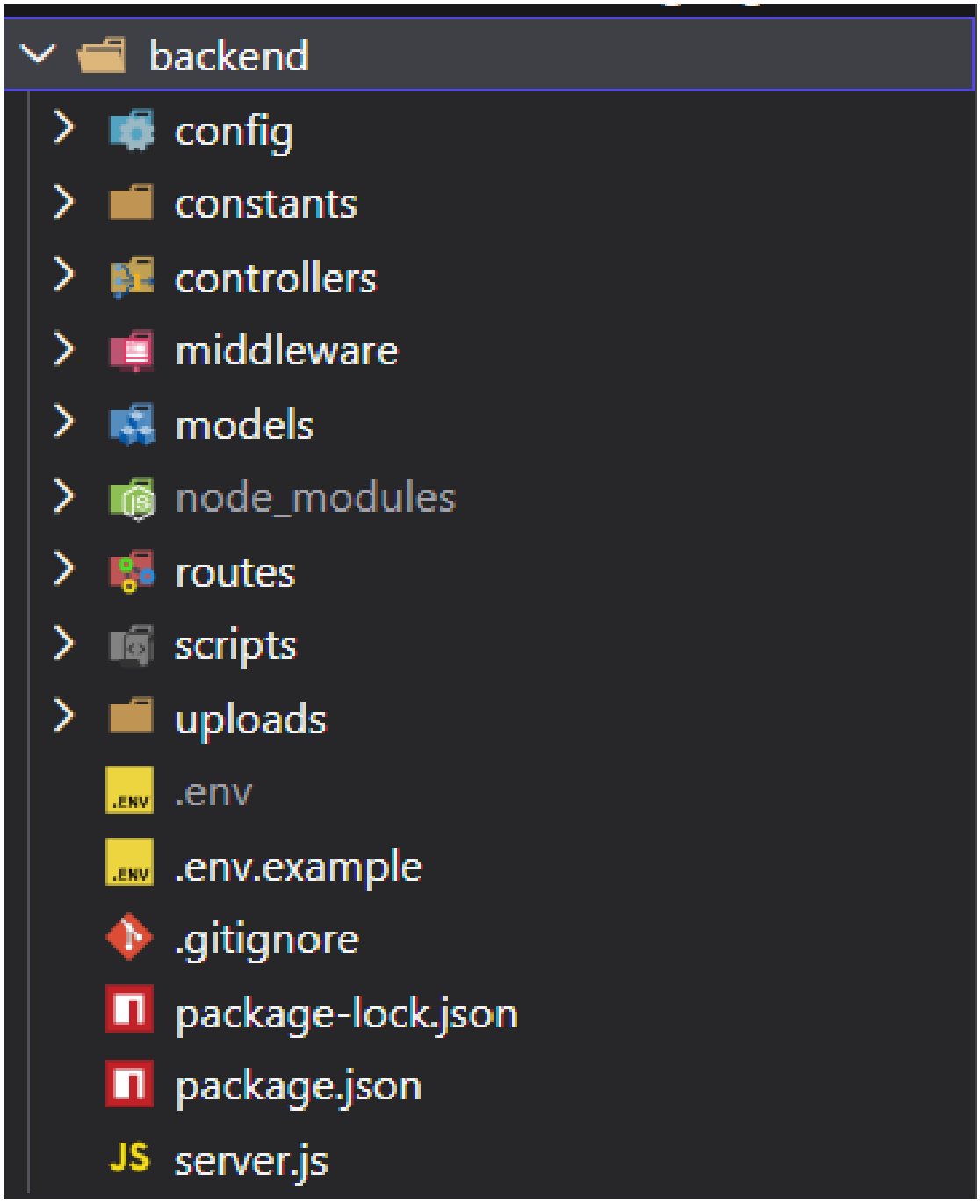

Figure 6: Backend setup steps and development server configuration

```
cd backend
npm install
npm run dev
```

The backend requires environment configuration for MongoDB, JWT, Cloudinary, and email settings. An administrator account can be seeded using:

```
npm run seed:admin
```

10.4 Environment Variables

Backend environment

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/
JWT_SECRET=your_jwt_secret
JWT_EXPIRE=7d

CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=465
EMAIL_USER=your_email
EMAIL_PASS=your_email_app_password

FRONTEND_URL=http://localhost:5173
```

Frontend environment

```
VITE_API_BASE_URL=/api
```

11 Backend Development

11.1 Backend Structure

The backend is divided into configuration, controllers, middleware, models, routes, and scripts.

```
backend/
|-- .env.example
|-- package.json
|-- server.js
|-- config/
|   |-- cloudinary.js
|   |-- db.js
|   `-- mail.js
|-- constants/
|   `-- departments.js
|-- controllers/
|   |-- authController.js
|   |-- documentController.js
|   |-- logController.js
|   |-- userController.js
|   `-- workflowController.js
|-- middleware/
|   |-- authMiddleware.js
|   |-- upload.js
```

```
|   `-- validation.js
|-- models/
|   |-- Document.js
|   |-- DocumentLog.js
|   |-- User.js
|   `-- Workflow.js
|-- routes/
|   |-- authRoutes.js
|   |-- documentRoutes.js
|   |-- logRoutes.js
|   |-- userRoutes.js
|   `-- workflowRoutes.js
|-- scripts/
|   `-- seedAdmin.js
`-- uploads/
    `-- .gitkeep
```

11.2 Backend Modules

- `authController.js`: Handles registration, login, OTP verification, and profile retrieval.
- `documentController.js`: Handles document upload, listing, detail retrieval, metadata updates, and deletion.
- `workflowController.js`: Handles forwarding, approval, and rejection actions.
- `userController.js`: Handles user management operations for authorized users.
- `logController.js`: Retrieves and exposes audit history.
- `authMiddleware.js`: Protects routes by validating JWT tokens and access level.
- `upload.js`: Validates file type and size during upload handling.

12 Database Development

12.1 MongoDB Configuration

MongoDB is used as the primary database because it provides flexible schema-based storage through Mongoose. The backend first establishes a database connection before processing any application request.

```
npm install mongoose
```

Once configured, the application connects to MongoDB using a dedicated database setup file inside the backend configuration folder.

12.2 Schemas and Models

The database design is implemented directly through Mongoose schemas. Each schema reflects one part of the document lifecycle: user management, document storage, workflow tracking, and audit logging.

12.2.1 `User.js`

The `User` model stores identity, login, role, and department information for every account in the system.

- `username`: Required, unique, trimmed, lowercase string with minimum length 3.
- `name`: Required display name of the user.
- `email`: Required, unique, trimmed, lowercase email address.
- `password`: Required password field stored in hashed form.
- `role`: Enumerated as `employee`, `manager`, or `admin`; default is `employee`.
- `department`: Enumerated department value derived from `DEPARTMENT_OPTIONS`; default is empty.
- `otp` and `otpExpires`: Temporary fields used during OTP-based authentication.
- `timestamps`: Automatically stores `createdAt` and `updatedAt`.

Before saving a user, the schema uses a `pre("save")` middleware hook to hash the password with `bcryptjs`. The model also provides a `matchPassword()` method that compares a plaintext password with the stored hash during login.

12.2.2 `Document.js`

The `Document` model stores the main metadata and ownership details for each uploaded file.

- `title`: Required trimmed string representing the document title.
- `description`: Optional trimmed text description; defaults to empty.
- `fileUrl`: Stores the final URL of the uploaded file.
- `cloudinaryPublicId`: Stores the Cloudinary asset identifier for file management.
- `uploadedBy`: Required `ObjectId` reference to the `User` model.
- `department`: Enumerated department string used for routing and visibility.
- `status`: Enumerated as `Submitted`, `Under Review`, `Approved`, or `Rejected`; default is `Submitted`.
- `remarks`: Optional remarks field for workflow comments and updates.
- `timestamps`: Automatically stores `createdAt` and `updatedAt`.

This schema captures both the descriptive information of a document and the latest state visible to employees, managers, and administrators.

12.2.3 `Workflow.js`

The `Workflow` model represents the live processing state of each document.

- `documentId`: Required `ObjectId` reference to the related `Document`.

- `currentStage`: Enumerated workflow stage with the same values used by the document status field.
- `assignedTo`: Optional `ObjectId` reference to the `User` currently responsible for review.
- `timestamps`: Automatically stores `createdAt` and `updatedAt`.

This model isolates routing information from document metadata so that the current assignee and review stage can be managed cleanly.

12.2.4 `DocumentLog.js`

The `DocumentLog` model stores the audit trail for every important workflow action.

- `documentId`: Required `ObjectId` reference to the affected document.
- `updatedBy`: Required `ObjectId` reference to the user who performed the action.
- `action`: Required enumerated string with values `Upload`, `Forward`, `Approve`, and `Reject`.
- `timestamp`: Stores the exact time of the action and defaults to `Date.now`.

This schema is important for traceability because it preserves a permanent history even when the current workflow stage changes later.

12.3 Relationships and Database Integrity

The implemented schema maintains clear links between entities using `ObjectId` references.

- One `User` can upload many `Document` records through `Document.uploadedBy`.
- One `Document` has one current `Workflow` record through `Workflow.documentId`.
- One `Document` can have many `DocumentLog` entries through `DocumentLog.documentId`.
- One `User` can appear in many log records through `DocumentLog.updatedBy`.
- One `User` can be assigned to many workflow records through `Workflow.assignedTo`, while the field may also remain null when unassigned.

Together, these schemas allow the application to preserve document ownership, workflow state, approval history, and user accountability in a structured and auditable form.

13 Front-End Development

13.1 Frontend Structure

The frontend is built with React and organized into reusable components, pages, context utilities, constants, services, and global state files.

```
frontend/
|-- package.json
```

```
|-- vite.config.js
|-- tailwind.config.js
|-- postcss.config.js
|-- index.html
`-- src/
    |-- App.jsx
    |-- main.jsx
    |-- index.css
    |-- components/
    |   |-- Chatbot.jsx
    |   |-- Navbar.jsx
    |   |-- ProtectedRoute.jsx
    |   |-- Sidebar.jsx
    |   `-- StatusBadge.jsx
    |-- constants/
    |   `-- departments.js
    |-- context/
    |   `-- AuthContext.jsx
    |-- pages/
    |   |-- AdminPanel.jsx
    |   |-- Dashboard.jsx
    |   |-- DocumentDetail.jsx
    |   |-- Landing.jsx
    |   |-- Login.jsx
    |   |-- Register.jsx
    |   `-- UploadDocument.jsx
    |-- services/
    |   `-- api.js
    `-- store/
        |-- documentFiltersSlice.js
        `-- index.js
```

13.2 Frontend Pages and Components

- `Landing.jsx`: Introduces the platform.
- `Login.jsx` and `Register.jsx`: Support authentication and account creation.
- `Dashboard.jsx`: Displays searchable and filterable document lists.
- `UploadDocument.jsx`: Provides document upload functionality.
- `DocumentDetail.jsx`: Shows metadata, workflow state, and audit history.
- `AdminPanel.jsx`: Supports user management for administrators.
- `ProtectedRoute.jsx`: Restricts access to authenticated users.
- `StatusBadge.jsx`: Displays document status consistently across the UI.

14 API Overview

The backend provides REST APIs for authentication, users, documents, workflows, and logs. Protected routes require the following header.

```
Authorization: Bearer <token>
```

14.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user.
POST	/api/auth/login	Validate credentials and send OTP.
POST	/api/auth/verify-otp	Verify OTP and issue JWT.
GET	/api/auth/profile	Get current logged-in user.
GET	/api/auth/users	Get all users; admin only.

14.2 Document Endpoints

Method	Endpoint	Description
POST	/api/documents	Upload a document.
GET	/api/documents	List visible documents with filters.
GET	/api/documents/:id	Get one document with workflow details.
PUT	/api/documents/:id	Update document metadata or remarks.
DELETE	/api/documents/:id	Delete a document; owner or admin only.
GET	/api/documents/:docId/logs	Retrieve audit history for a document.

14.3 Workflow and User Endpoints

Method	Endpoint	Description
GET	/api/workflow/:documentId	Get workflow details for a document.
PUT	/api/workflow/:documentId	Perform Forward, Approve, or Reject.
GET	/api/users/assignable	Get assignable users for workflow routing.
GET	/api/users	Get all users; admin only.

15 Advantages of the Implemented System

The Document Tracking System provides several practical benefits.

- It centralizes documents and eliminates scattered review processes.

- It improves accountability through role-based access and audit history.
- It reduces manual follow-up by clearly showing workflow status.
- It supports secure access through OTP login and JWT protection.
- It improves operational speed with dashboard filtering and direct workflow actions.
- It supports scalable file storage using Cloudinary and structured data storage using MongoDB.

16 Limitations and Future Scope

Although the current implementation is effective, several extensions can strengthen the system further.

- multi-stage approval chains with configurable rules;
- due dates, reminders, and escalation workflows;
- analytics dashboards for workload and turnaround time;
- downloadable reports and audit exports;
- file versioning and change history; and
- stronger deployment monitoring and role customization.

17 Output Screens

This section is reserved for user interface screenshots of the implemented system. Replace the placeholder image filenames below with the actual screenshot filenames when the final images are ready.

17.1 Landing Page

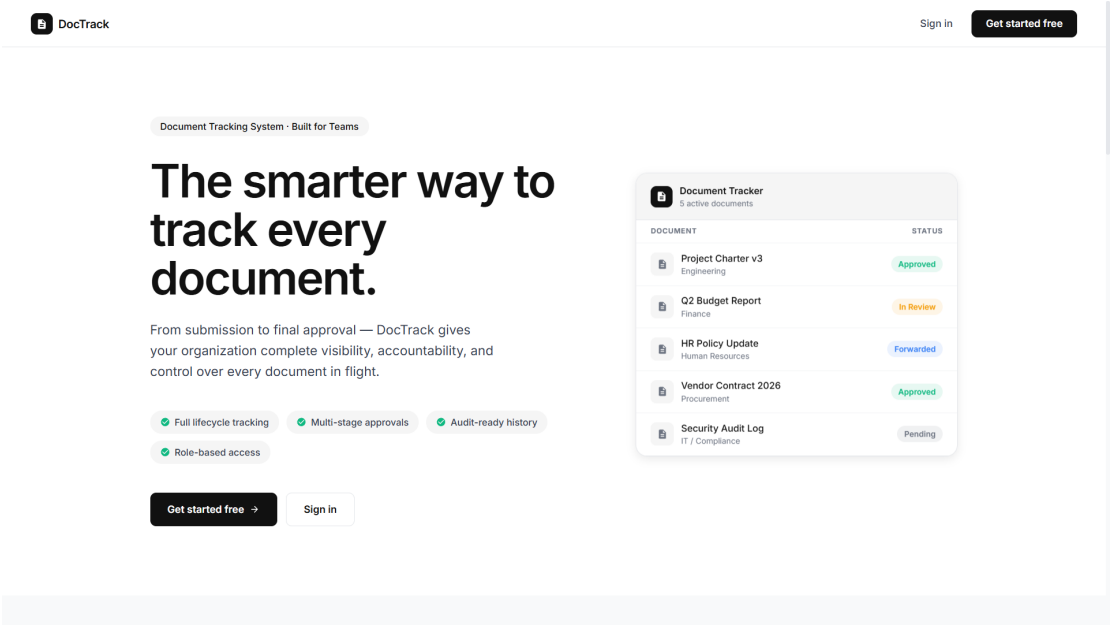

Figure 7: Landing page of the Document Tracking System

17.2 Login

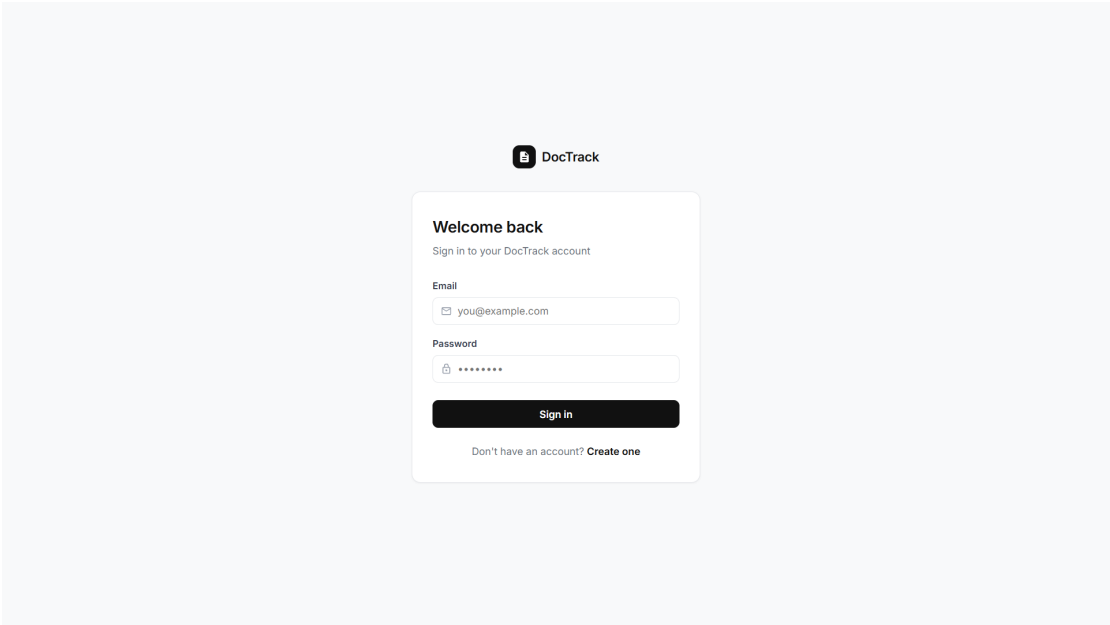

Figure 8: Login screen with OTP-based authentication flow

17.3 Signup

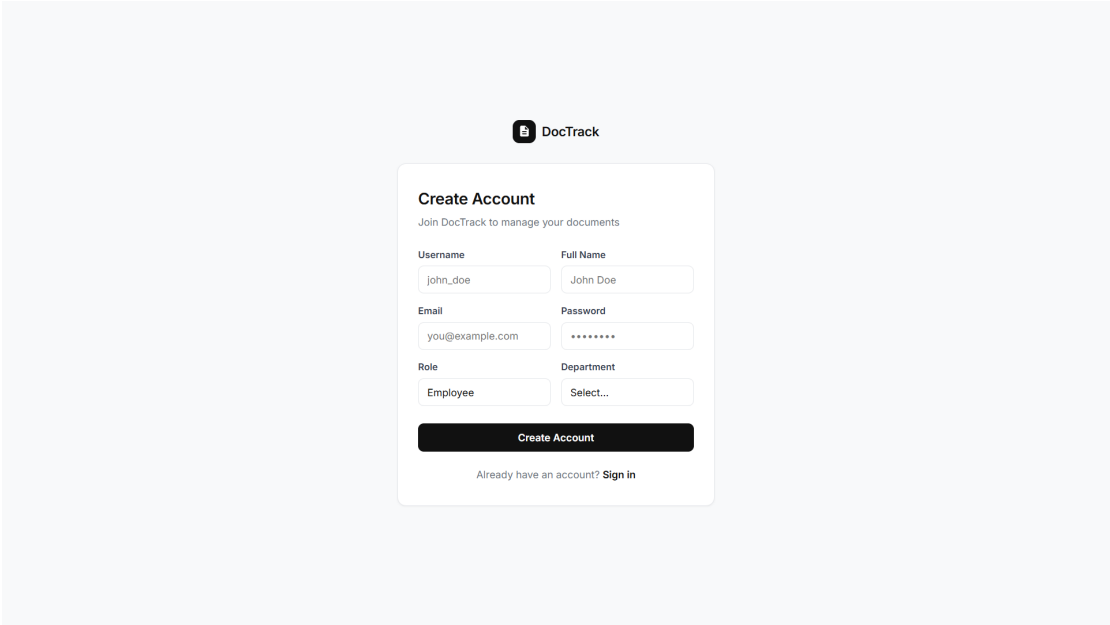

Figure 9: Signup or registration page for new users

17.4 Dashboard

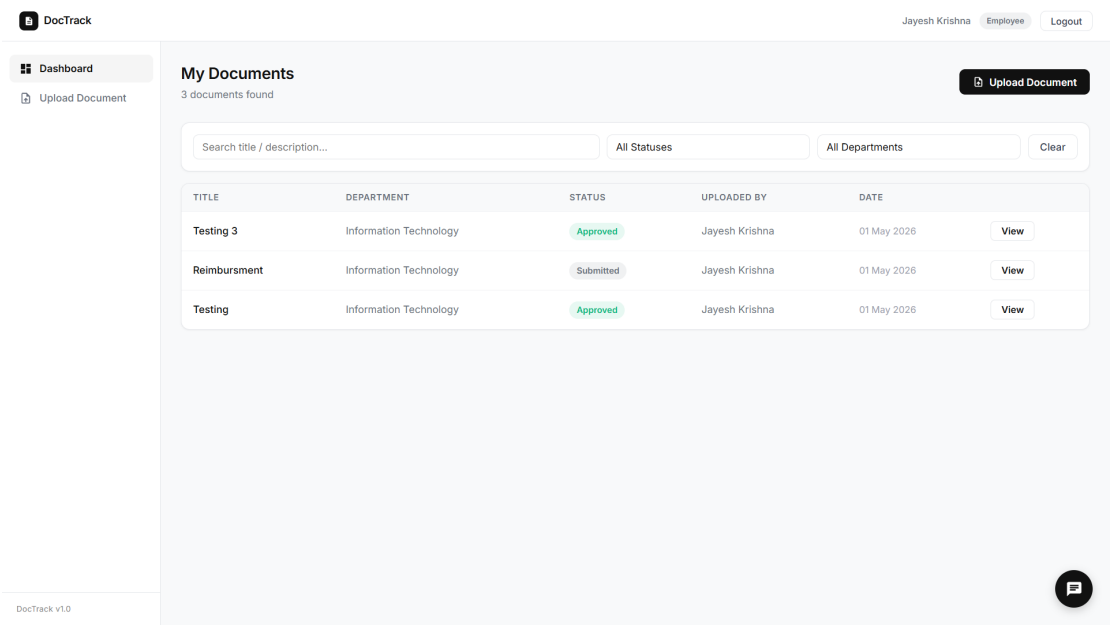

Figure 10: Dashboard view showing visible documents and filters

17.5 Upload Document

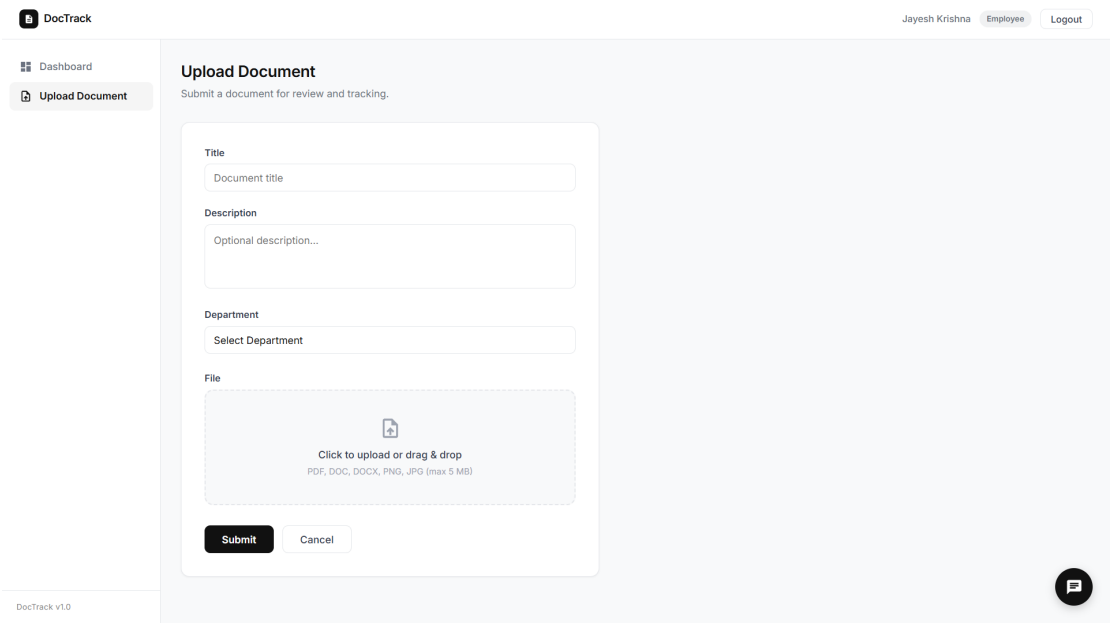

Figure 11: Document upload page used to submit new files

17.6 Chatbot

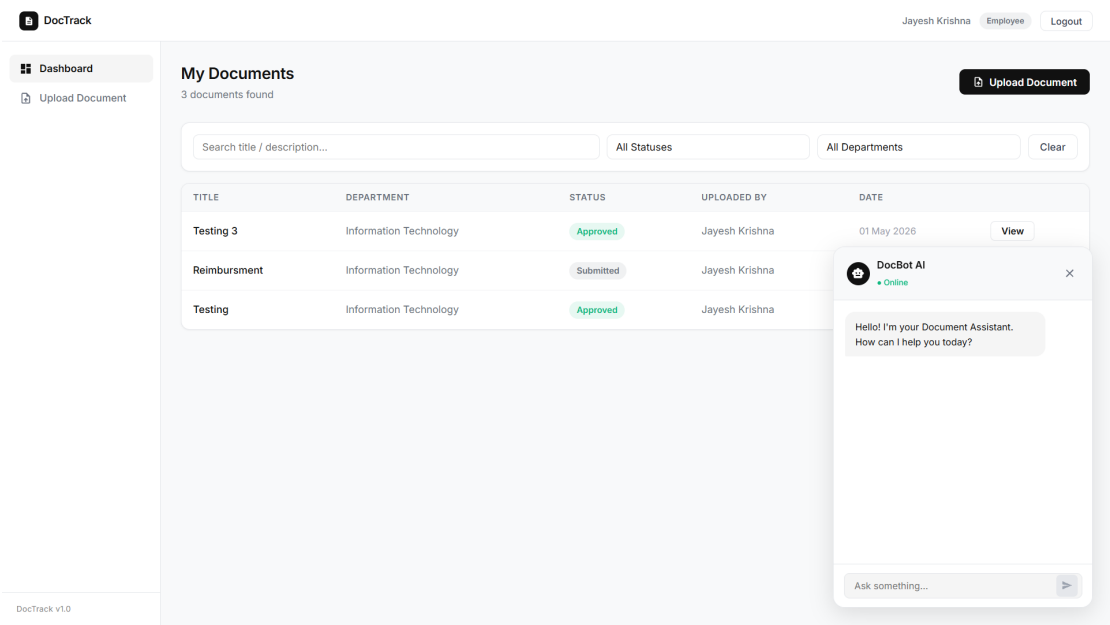

Figure 12: Integrated chatbot interface of the application

17.7 Admin Panel

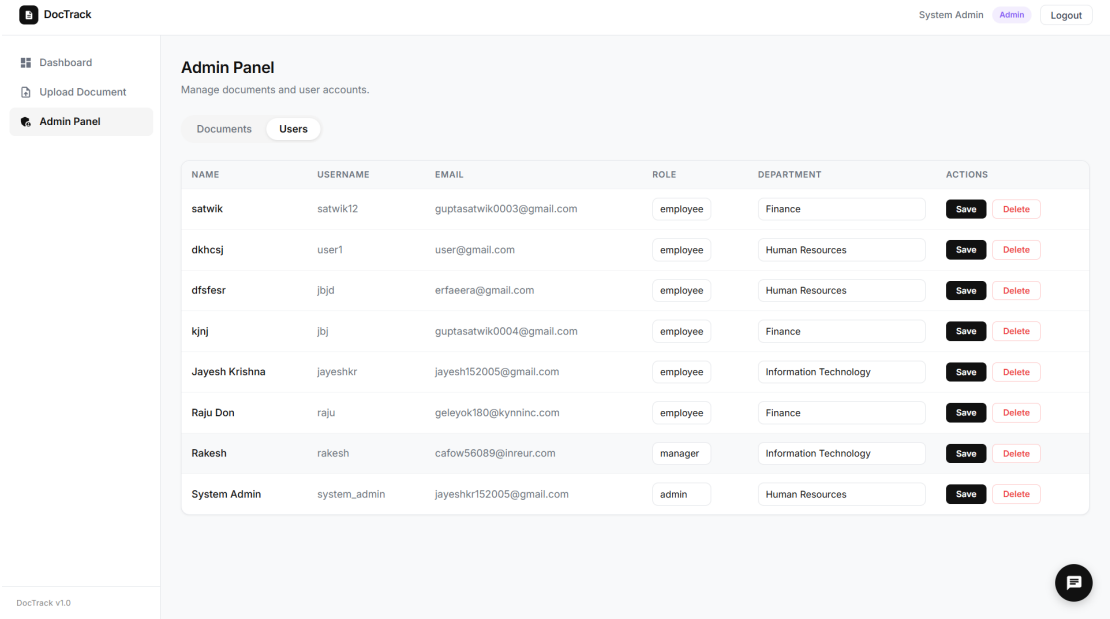

Figure 13: Administrative interface for user and system management

17.8 Document Detail

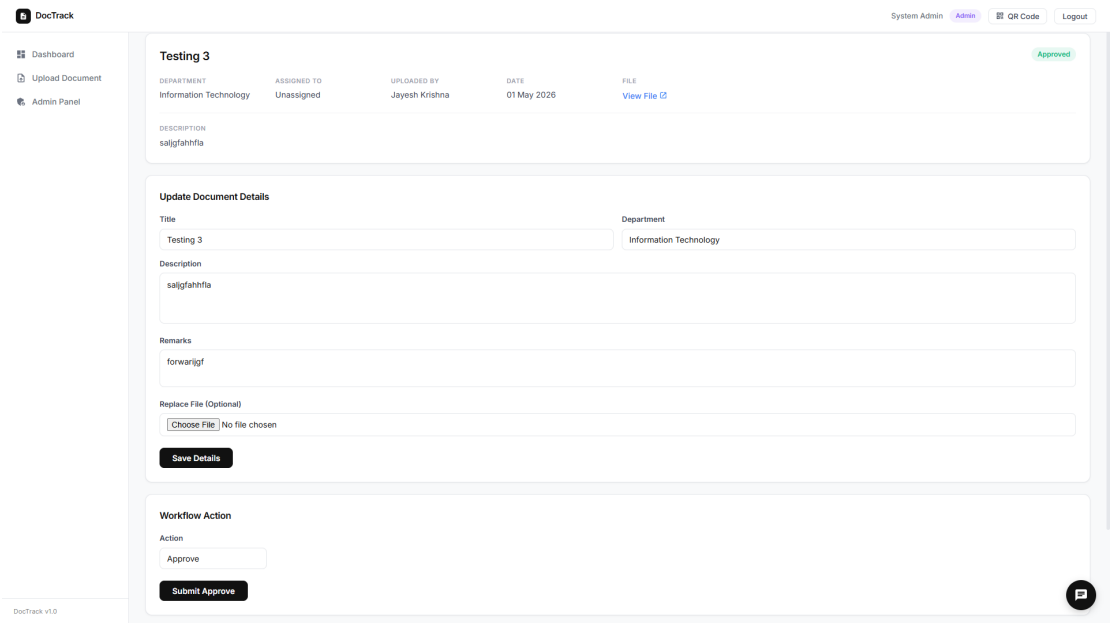

Figure 14: Detailed document view with workflow and audit information

18 Conclusion

The Document Tracking System is a practical and scalable solution for managing documents across departments. It combines secure authentication, structured workflow processing, centralized storage, role-based access control, and audit visibility into a single platform. By digitizing

document submission, forwarding, review, approval, and rejection, the system removes much of the ambiguity and delay present in traditional manual processes.

The project also demonstrates how frontend design, backend APIs, database modeling, and external service integration can work together to solve a real operational problem. With further enhancements, the platform can evolve into a more advanced enterprise-grade document management and approval system.